

A Half-Century of Progress



Since 1951, there have been thousands of new computers using a wide range of technologies and having widely varying capabilities. Figure e1.12.7 summarizes the key characteristics of some machines mentioned in this section and shows the dramatic changes that have occurred in just over 50 years. After adjusting for inflation, price/performance has improved by almost 100 billion in 55 years, or about 58% per year. Another way to say it is we've seen a factor of 10,000 improvement in cost and a factor of 10,000,000 improvement in performance.

Year	Name	Size (cu. ft.)	Power (watts)	Performance (adds/sec)	Memory (KB)	Price	Price/ performance vs. UNIVAC	Adjusted price (2007 \$)	Adjusted price/ performance vs. UNIVAC
1951	UNIVAC I	1,000	125,000	2,000	48	\$1,000,000	1	\$7,670,724	1
1964	IBM S/360 model 50	60	10,000	500,000	64	\$1,000,000	263	\$6,018,798	319
1965	PDP-8	8	500	330,000	4	\$16,000	10,855	\$94,685	13,367
1976	Cray-1	58	60,000	166,000,000	32,000	\$4,000,000	21,842	\$13,509,798	47,127
1981	IBM PC	1	150	240,000	256	\$3,000	42,105	\$6,859	134,208
1991	HP 9000/ model 750	2	500	50,000,000	16,384	\$7,400	3,556,188	\$11,807	16,241,889
1996	Intel PPro PC (200 MHz)	2	500	400,000,000	16,384	\$4,400	47,846,890	\$6,211	247,021,234
2003	Intel Pentium 4 PC (3.0 GHz)	2	500	6,000,000,000	262,144	\$1,600	1,875,000,000	\$2,009	11,451,750,000
2007	AMD Barcelona PC (2.5 GHz)	2	250	20,000,000,000	2,097,152	\$800	12,500,000,000	\$800	95,884,051,042

FIGURE e1.12.7 Characteristics of key commercial computers since 1950, in actual dollars and in 2007 dollars adjusted for inflation. The last row assumes we can fully utilize the potential performance of the four cores in Barcelona. In contrast to Figure e1.12.3, here the price of the IBM S/360 model 50 includes I/O devices. (Source: *The Computer History Museum and Producer Price Index for Industrial Commodities.*)

Readers interested in computer history should consult *Annals of the History of Computing*, a journal devoted to the history of computing. Several books describing the early days of computing have also appeared, many written by the pioneers including Goldstine [1972], Metropolis et al. [1980], and Wilkes [1985].

Further Reading

Barroso, L. and U. Hölzle [2007]. “The case for energy-proportional computing”, *IEEE Computer* December.

A plea to change the nature of computer components so that they use much less power when lightly utilized.

Bell, C. G. [1996]. *Computer Pioneers and Pioneer Computers*, ACM and the Computer Museum, videotapes.

Two videotapes on the history of computing, produced by Gordon and Gwen Bell, including the following machines and their inventors: Harvard Mark-I, ENIAC, EDSAC, IAS machine, and many others.

Burks, A. W., H. H. Goldstine, and J. von Neumann [1946]. “Preliminary discussion of the logical design of an electronic computing instrument,” Report to the U.S. Army Ordnance Department, p. 1; also appears in *Papers of John von Neumann*, W. Aspray and A. Burks (Eds.), MIT Press, Cambridge, MA, and Tomash Publishers, Los Angeles, 1987, 97–146.

A classic paper explaining computer hardware and software before the first stored-program computer was built. We quote extensively from it in Chapter 3. It simultaneously explained computers to the world and was a source of controversy because the first draft did not give credit to Eckert and Mauchly.

Campbell-Kelly, M. and W. Aspray [1996]. *Computer: A History of the Information Machine*, Basic Books, New York.

Two historians chronicle the dramatic story. The New York Times calls it well written and authoritative.

Ceruzzi, P. F. [1998]. *A History of Modern Computing*, MIT Press, Cambridge, MA.

Contains a good description of the later history of computing: the integrated circuit and its impact, personal computers, UNIX, and the Internet.

Curnow, H. J. and B. A. Wichmann [1976]. “A synthetic benchmark”, *The Computer J.* 19(1):80.

Describes the first major synthetic benchmark, Whetstone, and how it was created.

Flemming, P. J. and J. J. Wallace [1986]. “How not to lie with statistics: The correct way to summarize benchmark results”, *Comm. ACM* 29:3 (March), 218–21.

Describes some of the underlying principles in using different means to summarize performance results.

Goldstine, H. H. [1972]. *The Computer: From Pascal to von Neumann*, Princeton University Press, Princeton, NJ.

A personal view of computing by one of the pioneers who worked with von Neumann.

Hayes, B. [2007]. “Computing in a parallel universe”, *American Scientist* Vol. 95(November–December):476–480.

An overview of the parallel computing challenge written for the layman.

Hennessy, J. L. and D. A. Patterson [2007]. *Chapter 1 of Computer Architecture: A Quantitative Approach*, fourth edition, Morgan Kaufmann Publishers, San Francisco.

Section 1.5 goes into more detail on power, Section 1.6 contains much more detail on the cost of integrated circuits and explains the reasons for the difference between price and cost, and Section 1.8 gives more details on evaluating performance.

Lampson, B. W. [1986]. “Personal distributed computing; The Alto and Ethernet software.” In ACM Conference on the History of Personal Workstations (January).

Thacker, C. R. [1986]. "Personal distributed computing: The Alto and Ethernet hardware," In ACM Conference on the History of Personal Workstations (January).

These two papers describe the software and hardware of the landmark Alto.

Metropolis, N., J. Howlett, and G.-C. Rota (Eds.) [1980]. *A History of Computing in the Twentieth Century*, Academic Press, New York.

A collection of essays that describe the people, software, computers, and laboratories involved in the first experimental and commercial computers. Most of the authors were personally involved in the projects. An excellent bibliography of early reports concludes this interesting book.

Public Broadcasting System [1992]. *The Machine That Changed the World*, videotapes.

These five 1-hour programs include rare footage and interviews with pioneers of the computer industry.

Slater, R. [1987]. *Portraits in Silicon*, MIT Press, Cambridge, MA.

Short biographies of 31 computer pioneers.

Stern, N. [1980]. "Who invented the first electronic digital computer?" *Annals of the History of Computing* 2:4 (October), 375–76.

A historian's perspective on Atanasoff versus Eckert and Mauchly.

Weicker, R. P. [1984]. "Dhrystone: a synthetic systems programming benchmark", *Communications of the ACM* 27(10):1013–1030.

Description of a synthetic benchmarking program for systems code.

Wilkes, M. V. [1985]. *Memoirs of a Computer Pioneer*, MIT Press, Cambridge, MA.

A personal view of computing by one of the pioneers.

- e. Library reserve desk
- f. Increasing the gate area on a CMOS transistor to decrease its switching time
- g. Adding electromagnetic aircraft catapults (which are electrically powered as opposed to current steam-powered models), allowed by the increased power generation offered by the new reactor technology
- h. Building self-driving cars whose control systems partially rely on existing sensor systems already installed into the base vehicle, such as lane departure systems and smart cruise control systems

1.3 [2] <§1.3> Describe the steps that transform a program written in a high-level language such as C into a representation that is directly executed by a computer processor.

1.4 [2] <§1.4> Assume a color display using 8 bits for each of the primary colors (red, green, blue) per pixel and a frame size of 1280×1024 .

- a. What is the minimum size in bytes of the frame buffer to store a frame?
- b. How long would it take, at a minimum, for the frame to be sent over a 100 Mbit/s network?

1.5 [4] <§1.6> Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

- a. Which processor has the highest performance expressed in instructions per second?
- b. If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
- c. We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

1.6 [20] <§1.6> Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (classes A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2.

Given a program with a dynamic instruction count of $1.0E6$ instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D, which is faster: P1 or P2?

- a. What is the global CPI for each implementation?
- b. Find the clock cycles required in both cases.

1.7 [15] <§1.6> Compilers can have a profound impact on the performance of an application. Assume that for a program, compiler A results in a dynamic instruction count of $1.0E9$ and has an execution time of 1.1 s, while compiler B results in a dynamic instruction count of $1.2E9$ and an execution time of 1.5 s.

- a. Find the average CPI for each program given that the processor has a clock cycle time of 1 ns.
- b. Assume the compiled programs run on two different processors. If the execution times on the two processors are the same, how much faster is the clock of the processor running compiler A's code versus the clock of the processor running compiler B's code?
- c. A new compiler is developed that uses only $6.0E8$ instructions and has an average CPI of 1.1 . What is the speedup of using this new compiler versus using compiler A or B on the original processor?

1.8 The Pentium 4 Prescott processor, released in 2004, had a clock rate of 3.6 GHz and voltage of 1.25 V. Assume that, on average, it consumed 10 W of static power and 90 W of dynamic power.

The Core i5 Ivy Bridge, released in 2012, has a clock rate of 3.4 GHz and voltage of 0.9 V. Assume that, on average, it consumed 30 W of static power and 40 W of dynamic power.

1.8.1 [5] <§1.7> For each processor find the average capacitive loads.

1.8.2 [5] <§1.7> Find the percentage of the total dissipated power comprised by static power and the ratio of static power to dynamic power for each technology.

1.8.3 [15] <§1.7> If the total dissipated power is to be reduced by 10% , how much should the voltage be reduced to maintain the same leakage current? Note: power is defined as the product of voltage and current.

1.9 Assume for arithmetic, load/store, and branch instructions, a processor has CPIs of 1 , 12 , and 5 , respectively. Also assume that on a single processor a program requires the execution of $2.56E9$ arithmetic instructions, $1.28E9$ load/store instructions, and 256 million branch instructions. Assume that each processor has a 2 GHz clock frequency.

Assume that, as the program is parallelized to run over multiple cores, the number of arithmetic and load/store instructions per processor is divided by $0.7 \times p$ (where p is the number of processors) but the number of branch instructions per processor remains the same.

1.9.1 [5] <§1.7> Find the total execution time for this program on 1 , 2 , 4 , and 8 processors, and show the relative speedup of the 2 , 4 , and 8 processors result relative to the single processor result.

1.9.2 [10] <§§1.6, 1.8> If the CPI of the arithmetic instructions was doubled, what would the impact be on the execution time of the program on 1, 2, 4, or 8 processors?

1.9.3 [10] <§§1.6, 1.8> To what should the CPI of load/store instructions be reduced in order for a single processor to match the performance of four processors using the original CPI values?

1.10 Assume a 15 cm diameter wafer has a cost of 12, contains 84 dies, and has 0.020 defects/cm². Assume a 20 cm diameter wafer has a cost of 15, contains 100 dies, and has 0.031 defects/cm².

1.10.1 [10] <§1.5> Find the yield for both wafers.

1.10.2 [5] <§1.5> Find the cost per die for both wafers.

1.10.3 [5] <§1.5> If the number of dies per wafer is increased by 10% and the defects per area unit increases by 15%, find the die area and yield.

1.10.4 [5] <§1.5> Assume a fabrication process improves the yield from 0.92 to 0.95. Find the defects per area unit for each version of the technology given a die area of 200 mm².

1.11 The results of the SPEC CPU2006 bzip2 benchmark running on an AMD Barcelona has an instruction count of 2.389E12, an execution time of 750 s, and a reference time of 9650 s.

1.11.1 [5] <§§1.6, 1.9> Find the CPI if the clock cycle time is 0.333 ns.

1.11.2 [5] <§1.9> Find the SPECratio.

1.11.3 [5] <§§1.6, 1.9> Find the increase in CPU time if the number of instructions of the benchmark is increased by 10% without affecting the CPI.

1.11.4 [5] <§§1.6, 1.9> Find the increase in CPU time if the number of instructions of the benchmark is increased by 10% and the CPI is increased by 5%.

1.11.5 [5] <§§1.6, 1.9> Find the change in the SPECratio for this change.

1.11.6 [10] <§1.6> Suppose that we are developing a new version of the AMD Barcelona processor with a 4 GHz clock rate. We have added some additional instructions to the instruction set in such a way that the number of instructions has been reduced by 15%. The execution time is reduced to 700 s and the new SPECratio is 13.7. Find the new CPI.

1.11.7 [10] <§1.6> This CPI value is larger than obtained in 1.11.1 as the clock rate was increased from 3 GHz to 4 GHz. Determine whether the increase in the CPI is similar to that of the clock rate. If they are dissimilar, why?

1.11.8 [5] <§1.6> By how much has the CPU time been reduced?

1.11.9 [10] <§1.6> For a second benchmark, libquantum, assume an execution time of 960 ns, CPI of 1.61, and clock rate of 3 GHz. If the execution time is reduced by an additional 10% without affecting the CPI and with a clock rate of 4 GHz, determine the number of instructions.

1.11.10 [10] <§1.6> Determine the clock rate required to give a further 10% reduction in CPU time while maintaining the number of instructions and with the CPI unchanged.

1.11.11 [10] <§1.6> Determine the clock rate if the CPI is reduced by 15% and the CPU time by 20% while the number of instructions is unchanged.

1.12 Section 1.10 cites as a pitfall the utilization of a subset of the performance equation as a performance metric. To illustrate this, consider the following two processors. P1 has a clock rate of 4 GHz, average CPI of 0.9, and requires the execution of 5.0E9 instructions. P2 has a clock rate of 3 GHz, an average CPI of 0.75, and requires the execution of 1.0E9 instructions.

1.12.1 [5] <§§1.6, 1.10> One usual fallacy is to consider the computer with the largest clock rate as having the highest performance. Check if this is true for P1 and P2.

1.12.2 [10] <§§1.6, 1.10> Another fallacy is to consider that the processor executing the largest number of instructions will need a larger CPU time. Considering that processor P1 is executing a sequence of 1.0E9 instructions and that the CPI of processors P1 and P2 do not change, determine the number of instructions that P2 can execute in the same time that P1 needs to execute 1.0E9 instructions.

1.12.3 [10] <§§1.6, 1.10> A common fallacy is to use MIPS (*millions of instructions per second*) to compare the performance of two different processors, and consider that the processor with the largest MIPS has the largest performance. Check if this is true for P1 and P2.

1.12.4 [10] <§1.10> Another common performance figure is MFLOPS (millions of floating-point operations per second), defined as

$$\text{MFLOPS} = \text{No. FP operations} / (\text{execution time} \times 1\text{E}6)$$

but this figure has the same problems as MIPS. Assume that 40% of the instructions executed on both P1 and P2 are floating-point instructions. Find the MFLOPS figures for the processors.

1.13 Another pitfall cited in Section 1.10 is expecting to improve the overall performance of a computer by improving only one aspect of the computer. Consider a computer running a program that requires 250 s, with 70 s spent executing FP instructions, 85 s executed L/S instructions, and 40 s spent executing branch instructions.

1.13.1 [5] <§1.10> By how much is the total time reduced if the time for FP operations is reduced by 20%?

1.13.2 [5] <§1.10> By how much is the time for INT operations reduced if the total time is reduced by 20%?

1.13.3 [5] <§1.10> Can the total time can be reduced by 20% by reducing only the time for branch instructions?

1.14 Assume a program requires the execution of 50×10^6 FP instructions, 110×10^6 INT instructions, 80×10^6 L/S instructions, and 16×10^6 branch instructions. The CPI for each type of instruction is 1, 1, 4, and 2, respectively. Assume that the processor has a 2 GHz clock rate.

1.14.1 [10] <§1.10> By how much must we improve the CPI of FP instructions if we want the program to run two times faster?

1.14.2 [10] <§1.10> By how much must we improve the CPI of L/S instructions if we want the program to run two times faster?

1.14.3 [5] <§1.10> By how much is the execution time of the program improved if the CPI of INT and FP instructions is reduced by 40% and the CPI of L/S and Branch is reduced by 30%?

1.15 [5] <§1.8> When a program is adapted to run on multiple processors in a multiprocessor system, the execution time on each processor is comprised of computing time and the overhead time required for locked critical sections and/or to send data from one processor to another.

Assume a program requires $t = 100$ s of execution time on one processor. When run p processors, each processor requires t/p s, as well as an additional 4 s of overhead, irrespective of the number of processors. Compute the per-processor execution time for 2, 4, 8, 16, 32, 64, and 128 processors. For each case, list the corresponding speedup relative to a single processor and the ratio between actual speedup versus ideal speedup (speedup if there was no overhead).

§1.1, page 10: Discussion questions: many answers are acceptable.

§1.4, page 24: DRAM memory: volatile, short access time of 50 to 70 nanoseconds, and cost per GB is \$5 to \$10. Disk memory: nonvolatile, access times are 100,000 to 400,000 times slower than DRAM, and cost per GB is 100 times cheaper than DRAM. Flash memory: nonvolatile, access times are 100 to 1000 times slower than DRAM, and cost per GB is 7 to 10 times cheaper than DRAM.

§1.5, page 28: 1, 3, and 4 are valid reasons. Answer 5 can be generally true because high volume can make the extra investment to reduce die size by, say, 10% a good economic decision, but it doesn't have to be true.

§1.6, page 33: 1. a: both, b: latency, c: neither. 7 seconds.

§1.6, page 40: b.

§1.10, page 51: a. Computer A has the higher MIPS rating. b. Computer B is faster.

**Answers to
Check Yourself**

2

*I speak Spanish
to God, Italian to
women, French to
men, and German
to my horse.*

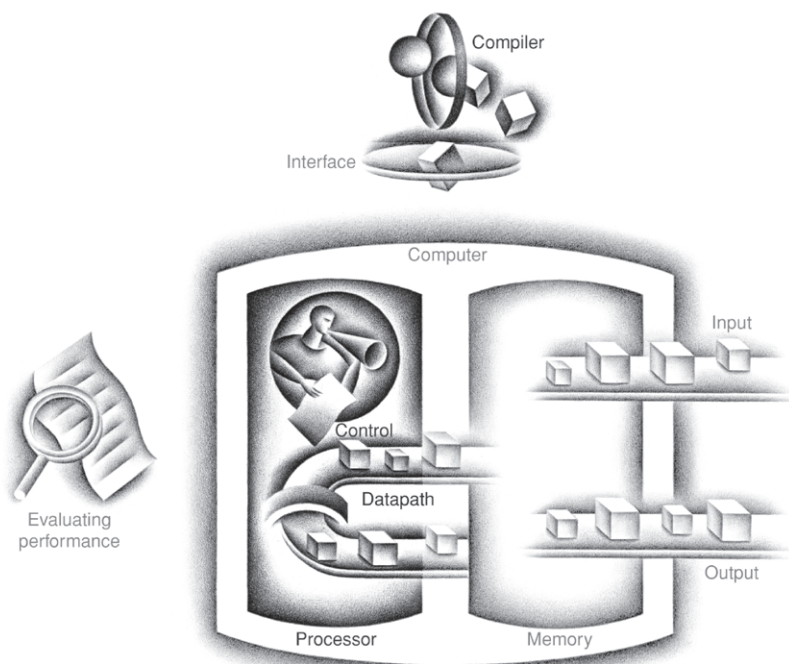
Charles V, Holy Roman Emperor
(1500–1558)

Instructions: Language of the Computer

- 2.1 Introduction** 62
- 2.2 Operations of the Computer Hardware** 63
- 2.3 Operands of the Computer Hardware** 67
- 2.4 Signed and Unsigned Numbers** 75
- 2.5 Representing Instructions in the
Computer** 82
- 2.6 Logical Operations** 90
- 2.7 Instructions for Making Decisions** 93

2.8	Supporting Procedures in Computer Hardware	100
2.9	Communicating with People	110
2.10	LEGv8 Addressing for Wide Immediates and Addresses	115
2.11	Parallelism and Instructions: Synchronization	125
2.12	Translating and Starting a Program	128
2.13	A C Sort Example to Put it All Together	137
2.14	Arrays versus Pointers	146
	2.15 Advanced Material: Compiling C and Interpreting Java	150
2.16	Real Stuff: MIPS Instructions	150
2.17	Real Stuff: ARMv7 (32-bit) Instructions	152
2.18	Real Stuff: x86 Instructions	154
2.19	Real Stuff: The Rest of the ARMv8 Instruction Set	163
2.20	Fallacies and Pitfalls	169
2.21	Concluding Remarks	171
	2.22 Historical Perspective and Further Reading	173
2.23	Exercises	174

The Five Classic Components of a Computer



2.1 Introduction

instruction set The vocabulary of commands understood by a given architecture.

To command a computer's hardware, you must speak its language. The words of a computer's language are called *instructions*, and its vocabulary is called an **instruction set**. In this chapter, you will see the instruction set of a real computer, both in the form written by people and in the form read by the computer. We introduce instructions in a top-down fashion. Starting from a notation that looks like a restricted programming language, we refine it step-by-step until you see the actual language of a real computer. [Chapter 3](#) continues our downward descent, unveiling the hardware for arithmetic and the representation of floating-point numbers.

You might think that the languages of computers would be as diverse as those of people, but in reality, computer languages are quite similar, more like regional dialects than independent languages. Hence, once you learn one, it is easy to pick up others.

The chosen instruction set is ARMv8, which comes from ARM Holdings plc and was announced in 2011. For pedagogic reasons, we will use a subset of ARMv8 instructions in this book. We will use the term ARMv8 when talking about the original full instruction set, and LEGv8 when referring to the teaching subset, which is of course based on ARM's ARMv8 instruction set. (LEGv8 is intended as a pun on ARMv8, but it is also a backronym for "Lessen Extrinsic Garrulity.") We'll identify differences between the two in the elaborations. Note that this chapter and several others have a section to give an overview of the rest of the features in ARMv8 that are not in LEGv8 (see Sections 2.19, 3.8, and 5.14).

To demonstrate how easy it is to pick up other instruction sets, we will take a quick look at three other popular instruction sets.

1. MIPS is an elegant example of the instruction sets designed since the 1980s.
2. ARMv7 is an older instruction set also from ARM Holdings plc, but with 32-bit addresses instead of ARMv8's 64 bits. More than 14 billion chips with ARM processors were manufactured in 2015, making them the most popular instruction sets in the world. Ironically, in the authors' opinion, and as shall be seen, ARMv8 is closer to MIPS than it is to ARMv7.
3. The final example is the Intel x86, which powers both the PC and the Cloud of the post-PC era.

This similarity of instruction sets occurs because all computers are constructed from hardware technologies based on similar underlying principles and because there are a few basic operations that all computers must provide. Moreover, computer designers have a common goal: to find a language that makes it easy to build the hardware and the compiler while maximizing performance and minimizing cost and energy. This goal is time-honored; the following quote was written before you could buy a computer, and it is as true today as it was in 1947:

It is easy to see by formal-logical methods that there exist certain [instruction sets] that are in abstract adequate to control and cause the execution of any

sequence of operations.... The really decisive considerations from the present point of view, in selecting an [instruction set], are more of a practical nature: simplicity of the equipment demanded by the [instruction set], and the clarity of its application to the actually important problems together with the speed of its handling of those problems.

Burks, Goldstine, and von Neumann, 1947

The “simplicity of the equipment” is as valuable a consideration for today’s computers as it was for those of the 1950s. The goal of this chapter is to teach an instruction set that follows this advice, showing both how it is represented in hardware and the relationship between high-level programming languages and this more primitive one. Our examples are in the C programming language; [Section 2.15](#) shows how these would change for an object-oriented language like Java.

By learning how to represent instructions, you will also discover the secret of computing: the **stored-program concept**. Moreover, you will exercise your “foreign language” skills by writing programs in the language of the computer and running them on the simulator that comes with this book. You will also see the impact of programming languages and compiler optimization on performance. We conclude with a look at the historical evolution of instruction sets and an overview of other computer dialects.

We reveal our first instruction set a piece at a time, giving the rationale along with the computer structures. This top-down, step-by-step tutorial weaves the components with their explanations, making the computer’s language more palatable. [Figure 2.1](#) gives a sneak preview of the instruction set covered in this chapter.

stored-program concept

The idea that instructions and data of many types can be stored in memory as numbers and thus be easy to change, leading to the stored-program computer.

2.2 Operations of the Computer Hardware

Every computer must be able to perform arithmetic. The LEGv8 assembly language notation

```
ADD a, b, c
```

instructs a computer to add the two variables *b* and *c* and to put their sum in *a*.

This notation is rigid in that each LEGv8 arithmetic instruction performs only one operation and must always have exactly three variables. For example, suppose we want to place the sum of four variables *b*, *c*, *d*, and *e* into variable *a*. (In this section, we are being deliberately vague about what a “variable” is; in the next section, we’ll explain in detail.)

The following sequence of instructions adds the four variables:

```
ADD a, b, c      // The sum of b and c is placed in a
ADD a, a, d      // The sum of b, c, and d is now in a
ADD a, a, e      // The sum of b, c, d, and e is now in a
```

Thus, it takes three instructions to sum the four variables.

There must certainly be instructions for performing the fundamental arithmetic operations.

Burks, Goldstine, and von Neumann, 1947

LEGv8 operands

Name	Example	Comments
32 registers	X0-X30, XZR	Fast locations for data. In LEGv8, data must be in registers to perform arithmetic, register XZR always equals 0.
2^{62} memory words	Memory[0], Memory[4], . . . , Memory[4,611,686,018,427,387,904]	Accessed only by data transfer instructions. LEGv8 uses byte addresses, so sequential doubleword addresses differ by 8. Memory holds data structures, arrays, and spilled registers.

LEGv8 assembly language

Category	InstructionExample		Meaning	Comments
Arithmetic	add	ADD X1, X2, X3	$X1 = X2 + X3$	Three register operands
	subtract	SUB X1, X2, X3	$X1 = X2 - X3$	Three register operands
	add immediate	ADDI X1, X2, 20	$X1 = X2 + 20$	Used to add constants
	subtract immediate	SUBI X1, X2, 20	$X1 = X2 - 20$	Used to subtract constants
	add and set flags	ADDS X1, X2, X3	$X1 = X2 + X3$	Add, set condition codes
	subtract and set flags	SUBS X1, X2, X3	$X1 = X2 - X3$	Subtract, set condition codes
	add immediate and set flags	ADDIS X1, X2, 20	$X1 = X2 + 20$	Add constant, set condition codes
	subtract immediate and set flags	SUBIS X1, X2, 20	$X1 = X2 - 20$	Subtract constant, set condition codes
Data transfer	load register	LDUR X1, [X2,40]	$X1 = \text{Memory}[X2 + 40]$	Doubleword from memory to register
	store register	STUR X1, [X2,40]	$\text{Memory}[X2 + 40] = X1$	Doubleword from register to memory
	load signed word	LDURSW X1, [X2,40]	$X1 = \text{Memory}[X2 + 40]$	Word from memory to register
	store word	STURW X1, [X2,40]	$\text{Memory}[X2 + 40] = X1$	Word from register to memory
	load half	LDURH X1, [X2,40]	$X1 = \text{Memory}[X2 + 40]$	Halfword memory to register
	store half	STURH X1, [X2,40]	$\text{Memory}[X2 + 40] = X1$	Halfword register to memory
	load byte	LDURB X1, [X2,40]	$X1 = \text{Memory}[X2 + 40]$	Byte from memory to register
	store byte	STURB X1, [X2,40]	$\text{Memory}[X2 + 40] = X1$	Byte from register to memory
	load exclusive register	LDXR X1, [X2,0]	$X1 = \text{Memory}[X2]$	Load; 1st half of atomic swap
	store exclusive register	STXR X1, X3 [X2]	$\text{Memory}[X2]=X1; X3=0 \text{ or } 1$	Store; 2nd half of atomic swap
	move wide with zero	MOVZ X1,20, LSL 0	$X1 = 20 \text{ or } 20 * 2^{16} \text{ or } 20 * 2^{32} \text{ or } 20 * 2^{48}$	Loads 16-bit constant, rest zeros
	move wide with keep	MOVK X1,20, LSL 0	$X1 = 20 \text{ or } 20 * 2^{16} \text{ or } 20 * 2^{32} \text{ or } 20 * 2^{48}$	Loads 16-bit constant, rest unchanged

FIGURE 2.1 LEGv8 assembly language revealed in this chapter. This information is also found in Column 1 of the LEGv8 Reference Data Card at the front of this book.

Logical	and	AND X1, X2, X3	X1 = X2 & X3	Three reg. operands; bit-by-bit AND
	inclusive or	ORR X1, X2, X3	X1 = X2 X3	Three reg. operands; bit-by-bit OR
	exclusive or	EOR X1, X2, X3	X1 = X2 ^ X3	Three reg. operands; bit-by-bit XOR
	and immediate	ANDI X1, X2, 20	X1 = X2 & 20	Bit-by-bit AND reg. with constant
	inclusive or immediate	ORRI X1, X2, 20	X1 = X2 20	Bit-by-bit OR reg. with constant
	exclusive or immediate	EORI X1, X2, 20	X1 = X2 ^ 20	Bit-by-bit XOR reg. with constant
	logical shift left	LSL X1, X2, 10	X1 = X2 << 10	Shift left by constant
	logical shift right	LSR X1, X2, 10	X1 = X2 >> 10	Shift right by constant
Conditional branch	compare and branch on equal 0	CBZ X1, 25	if (X1 == 0) go to PC + 100	Equal 0 test; PC-relative branch
	compare and branch on not equal 0	CBNZ X1, 25	if (X1 != 0) go to PC + 100	Not equal 0 test; PC-relative branch
	branch conditionally	B.cond 25	if (condition true) go to PC + 100	Test condition codes; if true, branch
Unconditional branch	branch	B 2500	go to PC + 10000	Branch to target address; PC-relative
	branch to register	BR X30	go to X30	For switch, procedure return
	branch with link	BL 2500	X30 = PC + 4; PC + 10000	For procedure call PC-relative

FIGURE 2.1 (Continued).

The words to the right of the double slashes (//) on each line above are *comments* for the human reader, so the computer ignores them. Note that unlike other programming languages, each line of this language can contain at most one instruction. Another difference from C is that comments always terminate at the end of a line.

The natural number of operands for an operation like addition is three: the two numbers being added together and a place to put the sum. Requiring every instruction to have exactly three operands, no more and no less, conforms to the philosophy of keeping the hardware simple: hardware for a variable number of operands is more complicated than hardware for a fixed number. This situation illustrates the first of three underlying principles of hardware design:

Design Principle 1: Simplicity favors regularity.

We can now show, in the two examples that follow, the relationship of programs written in higher-level programming languages to programs in this more primitive notation.

EXAMPLE**Compiling Two C Assignment Statements into LEGv8**

This segment of a C program contains the five variables *a*, *b*, *c*, *d*, and *e*. Since Java evolved from C, this example and the next few work for either high-level programming language:

```
a = b + c;  
d = a - e;
```

The *compiler* translates from C to LEGv8 assembly language instructions. Show the LEGv8 code produced by a compiler.

ANSWER

An LEGv8 instruction operates on two source operands and places the result in one destination operand. Hence, the two simple statements above compile directly into these two LEGv8 assembly language instructions:

```
ADD a, b, c  
SUB d, a, e
```

EXAMPLE**Compiling a Complex C Assignment into LEGv8**

A somewhat complicated statement contains the five variables *f*, *g*, *h*, *i*, and *j*:

```
f = (g + h) - (i + j);
```

What might a C compiler produce?

The compiler must break this statement into several assembly instructions, since only one operation is performed per LEGv8 instruction. The first LEGv8 instruction calculates the sum of *g* and *h*. We must place the result somewhere, so the compiler creates a temporary variable, called *t0*:

```
ADD t0,g,h // temporary variable t0 contains g + h
```

Although the next operation is subtract, we need to calculate the sum of *i* and *j* before we can subtract. Thus, the second instruction places the sum of *i* and *j* in another temporary variable created by the compiler, called *t1*:

```
ADD t1,i,j // temporary variable t1 contains i + j
```

Finally, the subtract instruction subtracts the second sum from the first and places the difference in the variable *f*, completing the compiled code:

```
SUB f,t0,t1 // f gets t0 - t1, which is (g + h) - (i + j)
```

For a given function, which programming language likely takes the most lines of code? Put the three representations below in order.

**Check
Yourself**

1. Java
2. C
3. LEGv8 assembly language

Elaboration: To increase portability, Java was originally envisioned as relying on a software interpreter. The instruction set of this interpreter is called *Java bytecodes* (see  **Section 2.15**), which is quite different from the LEGv8 instruction set. To get performance close to the equivalent C program, Java systems today typically compile Java bytecodes into the native instruction sets like LEGv8. Because this compilation is normally done much later than for C programs, such Java compilers are often called *Just In Time* (JIT) compilers. Section 2.12 shows how JITs are used later than C compilers in the start-up process, and Section 2.13 shows the performance consequences of compiling versus interpreting Java programs.

2.3 Operands of the Computer Hardware

Unlike programs in high-level languages, the operands of arithmetic instructions are restricted; they must be from a limited number of special locations built directly in hardware called *registers*. Registers are primitives used in hardware design that are also visible to the programmer when the computer is completed, so you can think of registers as the bricks of computer construction. The size of a register in the LEGv8 architecture is 64 bits; groups of 64 bits occur so frequently that they are given the name **doubleword** in the LEGv8 architecture. (Another popular size is a group of 32 bits, called a **word** in the LEGv8 architecture.)

One major difference between the variables of a programming language and registers is the limited number of registers, typically 32 on current computers, like LEGv8. (See  **Section 2.22** for the history of the number of registers.) Thus, continuing in our top-down, stepwise evolution of the symbolic representation of the LEGv8 language, in this section we have added the restriction that the three operands of LEGv8 arithmetic instructions must each be chosen from one of the 32 64-bit registers.

The reason for the limit of 32 registers may be found in the second of our three underlying design principles of hardware technology:

Design Principle 2: Smaller is faster.

A very large number of registers may increase the clock cycle time simply because it takes electronic signals longer when they must travel farther.

Guidelines such as “smaller is faster” are not absolutes; 31 registers may not be faster than 32. Even so, the truth behind such observations causes computer designers to take them seriously. In this case, the designer must balance the craving of

doubleword Another natural unit of access in a computer, usually a group of 64 bits; corresponds to the size of a register in the LEGv8 architecture.

word A natural unit of access in a computer, usually a group of 32 bits.

programs for more registers with the designer's desire to keep the clock cycle fast. Another reason for not using more than 32 is the number of bits it would take in the instruction format, as Section 2.5 demonstrates.

Chapter 4 shows the central role that registers play in hardware construction; as we shall see in that chapter, effective use of registers is critical to program performance.

Although we could simply write instructions using numbers for registers, from 0 to 31, the LEGv8 convention is X followed by the number of the register, except for a few register names that we will cover later.

EXAMPLE

Compiling a C Assignment Using Registers

It is the compiler's job to associate program variables with registers. Take, for instance, the assignment statement from our earlier example:

```
f = (g + h) - (i + j);
```

The variables *f*, *g*, *h*, *i*, and *j* are assigned to the registers X19, X20, X21, X22, and X23, respectively. What is the compiled LEGv8 code?

ANSWER

The compiled program is very similar to the prior example, except we replace the variables with the register names mentioned above plus two temporary registers, X9 and X10, which correspond to the temporary variables above:

```
ADD X9,X20,X21 // register X9 contains g + h
ADD X10,X22,X23 // register X10 contains i + j
SUB X19,X9,X10 // f gets X9 - X10, which is (g + h) - (i + j)
```

Memory Operands

Programming languages have simple variables that contain single data elements, as in these examples, but they also have more complex data structures—arrays and structures. These composite data structures can contain many more data elements than there are registers in a computer. How can a computer represent and access such large structures?

Recall the five components of a computer introduced in Chapter 1 and repeated on page 61. The processor can keep only a small amount of data in registers, but computer memory contains billions of data elements. Hence, data structures (arrays and structures) are kept in memory.

As explained above, arithmetic operations occur only on registers in LEGv8 instructions; thus, LEGv8 must include instructions that transfer data between memory and registers. Such instructions are called **data transfer instructions**.

data transfer

instruction A command that moves data between memory and registers.

To access a word or doubleword in memory, the instruction must supply the memory **address**. Memory is just a large, single-dimensional array, with the address acting as the index to that array, starting at 0. For example, in Figure 2.2, the address of the third data element is 2, and the value of memory [2] is 10.

address A value used to delineate the location of a specific data element within a memory array.

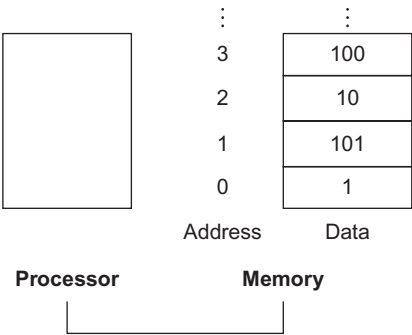


FIGURE 2.2 Memory addresses and contents of memory at those locations. If these elements were doublewords, these addresses would be incorrect, since LEGv8 actually uses byte addressing, with each doubleword representing 8 bytes. Figure 2.3 shows the correct memory addressing for sequential doubleword addresses.

The data transfer instruction that copies data from memory to a register is traditionally called *load*. The format of the load instruction is the name of the operation followed by the register to be loaded, then register and a constant used to access memory. The sum of the constant portion of the instruction and the contents of the second register forms the memory address. The real LEGv8 name for this instruction is LDUR, standing for *load register*.

Elaboration: The U in LDUR stands for *unscaled* immediate as opposite to *scaled* immediate, which we explain in Section 2.19.

Compiling an Assignment When an Operand Is in Memory

Let’s assume that A is an array of 100 doublewords and that the compiler has associated the variables g and h with the registers X20 and X21 as before. Let’s also assume that the starting address, or *base address*, of the array is in X22. Compile this C assignment statement:

```
g = h + A[8];
```

Although there is a single operation in this assignment statement, one of the operands is in memory, so we must first transfer A[8] to a register. The address

EXAMPLE

ANSWER

of this array element is the sum of the base of the array *A*, found in register *X22*, plus the number to select element 8. The data should be placed in a temporary register for use in the next instruction. Based on Figure 2.2, the first compiled instruction is

```
LDUR      X9,[X22,#8] // Temporary reg X9 gets A[8]
```

(We'll be making a slight adjustment to this instruction, but we'll use this simplified version for now.) The following instruction can operate on the value in *X9* (which equals *A[8]*) since it is in a register. The instruction must add *h* (contained in *X21*) to *A[8]* (contained in *X9*) and put the sum in the register corresponding to *g* (associated with *X20*):

```
ADD      X20,X21,X9 // g = h + A[8]
```

The register added to form the address (*X22*) is called the *base register*, and the constant in a data transfer instruction (8) is called the *offset*.

Hardware/ Software Interface

In addition to associating variables with registers, the compiler allocates data structures like arrays and structures to locations in memory. The compiler can then place the proper starting address into the data transfer instructions.

Since 8-bit *bytes* are useful in many programs, virtually all architectures today address individual bytes. Therefore, the address of a doubleword matches the address of one of the 8 bytes within the doubleword, and addresses of sequential doublewords differ by 8. For example, Figure 2.3 shows the actual LEGv8 addresses for the doublewords in Figure 2.2; the byte address of the third doubleword is 16.

Computers divide into those that use the address of the leftmost or “big end” byte as the doubleword address versus those that use the rightmost or “little end” byte. LEGv8 can work either as *big-endian* or *little-endian*. Since the order matters only if you access the identical data both as a doubleword and as eight bytes, few need to be aware of the “endianess”.

Byte addressing also affects the array index. To get the proper byte address in the code above, *the offset to be added to the base register X22 must be 8×8 , or 64*, so that the load address will select *A[8]* and not *A[8/8]*. (See the related *Pitfall* on page 171 of Section 2.20.)

The instruction complementary to load is traditionally called *store*; it copies data from a register to memory. The format of a store is similar to that of a load: the name of the operation, followed by the register to be stored, then the base register, and finally the offset to select the array element. Once again, the LEGv8 address is specified in part by a constant and in part by the contents of a register. The actual LEGv8 name is STUR, standing for *store register*.

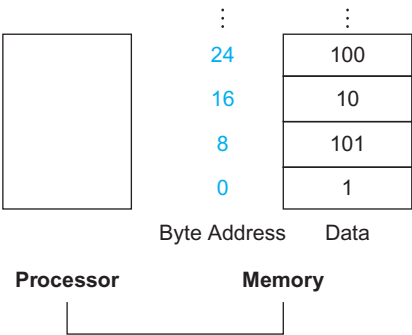


FIGURE 2.3 Actual LEGv8 memory addresses and contents of memory for those doublewords. The changed addresses are highlighted to contrast with Figure 2.2. Since LEGv8 addresses each byte, doubleword addresses are multiples of 8: there are 8 bytes in a doubleword.

Elaboration: In many architectures, words must start at addresses that are multiples of 4 and doublewords must start at addresses that are multiples of 8. This requirement is called an **alignment restriction**. (Chapter 4 suggests why alignment leads to faster data transfers.) ARMv8 and Intel x86 *do not* have alignment restrictions, but ARMv7 and MIPS do.

alignment restriction
A requirement that data be aligned in memory on natural boundaries.

Elaboration: It is not quite true that ARMv8 has no alignment restrictions. While it does support unaligned access to normal memory for most data transfer instructions, stack accesses and instruction fetches do have alignment restrictions.

As the addresses in loads and stores are binary numbers, we can see why the DRAM for main memory comes in binary sizes rather than in decimal sizes. That is, in gibibytes (2^{30}) or tebibytes (2^{40}), not in gigabytes (10^9) or terabytes (10^{12}); see Figure 1.1.

**Hardware/
Software
Interface**

Compiling Using Load and Store

Assume variable *h* is associated with register X21 and the base address of the array *A* is in X22. What is the LEGv8 assembly code for the C assignment statement below?

```
A[12] = h + A[8];
```

Although there is a single operation in the C statement, now two of the operands are in memory, so we need even more LEGv8 instructions. The first two instructions are the same as in the prior example, except this time we use

EXAMPLE

ANSWER

the proper offset for byte addressing in the load register instruction to select A[8], and the ADD instruction places the sum in X9:

```
LDUR X9, [X22,#64] // Temporary reg X9 gets A[8]
ADD  X9,X21,X9      // Temporary reg X9 gets h + A[8]
```

The final instruction stores the sum into A[12], using 96 (8×12) as the offset and register X22 as the base register.

```
STUR X9, [X22,#96] // Stores h + A[8] back into A[12]
```

Load register and store register are the instructions that copy doublewords between memory and registers in the ARMv8 architecture. Some brands of computers use other instructions along with load and store to transfer data. An architecture with such alternatives is the Intel x86, described in Section 2.18.

Hardware/ Software Interface

Many programs have more variables than computers have registers. Consequently, the compiler tries to keep the most frequently used variables in registers and places the rest in memory, using loads and stores to move variables between registers and memory. The process of putting less frequently used variables (or those needed later) into memory is called *spilling* registers.

The hardware principle relating size and speed suggests that memory must be slower than registers, since there are fewer registers. This suggestion is indeed the case; data accesses are faster if data are in registers instead of memory.

Moreover, data are more useful when in a register. A LEGv8 arithmetic instruction can read two registers, operate on them, and write the result. A LEGv8 data transfer instruction only reads one operand or writes one operand, without operating on it.

Thus, registers take less time to access *and* have higher throughput than memory, making data in registers both considerably faster to access and simpler to use. Accessing registers also uses much less energy than accessing memory. To achieve the highest performance and conserve energy, an instruction set architecture must have enough registers, and compilers must use registers efficiently.

Elaboration: Let's put the energy and performance of registers versus memory into perspective. Assuming 64-bit data, registers are roughly 200 times faster (0.25 vs. 50 nanoseconds) and are 10,000 times more energy efficient (0.1 vs. 1000 picoJoules) than DRAM in 2015. These large differences led to caches, which reduce the performance and energy penalties of going to memory (see [Chapter 5](#)).

Constant or Immediate Operands

Many times a program will use a constant in an operation—for example, incrementing an index to point to the next element of an array. In fact, more than half of the LEGv8 arithmetic instructions have a constant as an operand when running the SPEC CPU2006 benchmarks.

Using only the instructions we have seen so far, we would have to load a constant from memory to use one. (The constants would have been placed in memory when the program was loaded.) For example, to add the constant 4 to register X22, we could use the code

```
LDUR X9, [X20, AddrConstant4]    // X9 = constant 4
ADD  X22, X22, X9                 // X22 = X22 + X9 (X9 == 4)
```

assuming that `X20 + AddrConstant4` is the memory address of the constant 4.

An alternative that avoids the load instruction is to offer versions of the arithmetic instructions in which one operand is a constant. This quick add instruction with one constant operand is called *add immediate* or `ADDI`. To add 4 to register X22, we just write

```
ADDI    X22, X22, #4              // X22 = X22 + 4
```

Constant operands occur frequently, and by including constants inside arithmetic instructions, operations are much faster and use less energy than if constants were loaded from memory.

The constant zero has another role, which is to simplify the instruction set by offering useful variations. For example, the move operation is just an add instruction where one operand is zero. Hence, LEGv8 dedicates a register `XZR` to be hard-wired to the value zero. (It corresponds to register number 31.) Using frequency to justify the inclusions of constants is another example of the great idea from [Chapter 1](#) of making the **common case fast**.



COMMON CASE FAST

Given the importance of registers, what is the rate of increase in the number of registers in a chip over time?

1. Very fast: They increase as fast as **Moore's Law**, which predicts doubling the number of transistors on a chip every 18 months.
2. Very slow: Since programs are usually distributed in the language of the computer, there is inertia in instruction set architecture, and so the number of registers increases only as fast as new instruction sets become viable.

Elaboration: Although the LEGv8 registers in this book are 64 bits wide, the full ARMv8 instruction set has two execution states: *AArch32*, in which registers are 32 bits wide (see Section 2.19), and *AArch64*, which has a 64-bit wide register. The former supports the A32 and T32 instruction sets and the latter supports A64. In this chapter, we use a subset of A64 for LEGv8.

Check Yourself



MOORE'S LAW

Elaboration: The LEGv8 offset plus base register addressing is an excellent match to structures as well as arrays, since the register can point to the beginning of the structure and the offset can select the desired element. We'll see such an example in Section 2.13.

Elaboration: The register in the data transfer instructions was originally invented to hold an index of an array with the offset used for the starting address of an array. Thus, the base register is also called the *index register*. Today's memories are much larger, and the software model of data allocation is more sophisticated, so the base address of the array is normally passed in a register since it won't fit in the offset, as we shall see.

Elaboration: The migration from 32-bit address computers to 64-bit address computers left compiler writers a choice of the size of data types in C. Clearly, pointers should be 64 bits, but what about integers? Moreover, C has the data types `int`, `long int`, and `long long int`. The problems come from converting from one data type to another and having an unexpected overflow in C code that is not fully standard compliant, which unfortunately is not rare code. The table below shows the two popular options:

Operating System	pointers	int	long int	long long int
Microsoft Windows	64 bits	32 bits	32 bits	64 bits
Linux, Most Unix	64 bits	32 bits	64 bits	64 bits

While each compiler could have different choices, generally the compilers associated with each operating system make the same decision. To keep the examples simple, in this book we'll assume pointers are all 64 bits and declare all C integers as `long long int` to keep them the same size. We also follow C99 standard and declare variables used as indexes to arrays to be `size_t`, which guarantees they are the right size no matter how big the array. They typically declared the same as `long int`.

Elaboration: In the full ARMv8 instruction set, register 31 is `XZR` in most instructions but the stack point (`SP`) in others. We think it is confusing, so register 31 is always `XZR` in LEGv8 and `SP` is always register 28. Besides confusing the reader, it would also complicate datapath design in [Chapter 4](#) if register 31 meant 0 for some instructions and `SP` for others.

Elaboration: The full ARMv8 instruction set does not use the mnemonic `ADDI` when one of the operands is an immediate; it just uses `ADD`, and lets the assembler pick the proper opcode. We worry that it might be confusing to use the same mnemonic for both opcodes, so for teaching purposes LEGv8 distinguishes the two cases with different mnemonics.

2.4 Signed and Unsigned Numbers

First, let's quickly review how a computer represents numbers. Humans are taught to think in base 10, but numbers may be represented in any base. For example, 123 base 10 = 1111011 base 2.

Numbers are kept in computer hardware as a series of high and low electronic signals, and so they are considered base 2 numbers. (Just as base 10 numbers are called *decimal* numbers, base 2 numbers are called *binary* numbers.)

A single digit of a binary number is thus the “atom” of computing, since all information is composed of **binary digits** or *bits*. This fundamental building block can be one of two values, which can be thought of as several alternatives: high or low, on or off, true or false, or 1 or 0.

Generalizing the point, in any number base, the value of i th digit d is

$$d \times \text{Base}^i$$

where i starts at 0 and increases from right to left. This representation leads to an obvious way to number the bits in the doubleword: simply use the power of the base for that bit. We subscript decimal numbers with *ten* and binary numbers with *two*. For example,

1011_{two}

represents

$$\begin{aligned} & (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)_{\text{ten}} \\ &= (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1)_{\text{ten}} \\ &= 8 + 0 + 2 + 1_{\text{ten}} \\ &= 11_{\text{ten}} \end{aligned}$$

We number the bits 0, 1, 2, 3, ... from *right to left* in a doubleword. The drawing below shows the numbering of bits within an LEGv8 doubleword and the placement of the number 1011_{two}, (which we must unfortunately split in half to fit on the page of the book):

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48	47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

(64 bits wide, split into two 32-bit rows)

Since doublewords are drawn vertically as well as horizontally, leftmost and rightmost may be unclear. Hence, the phrase **least significant bit** is used to refer to the rightmost bit (bit 0 above) and **most significant bit** to the leftmost bit (bit 63).

binary digit Also called **binary bit**. One of the two numbers in base 2, 0 or 1, that are the components of information.

least significant bit The rightmost bit in an LEGv8 doubleword.

most significant bit The leftmost bit in an LEGv8 doubleword.

The LEGv8 doubleword is 64 bits long, so we can represent 2^{64} different 64-bit patterns. It is natural to let these combinations represent the numbers from 0 to $2^{64} - 1$ (18,446,774,073,709,551,615_{ten}):

```

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000two = 0ten
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000001two = 1ten
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000010two = 2ten
...
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111101two = 18,446,774,073,709,551,613ten
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110two = 18,446,744,073,709,551,614ten
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111two = 18,446,744,073,709,551,615ten

```

That is, 64-bit binary numbers can be represented in terms of the bit value times a power of 2 (here x_i means the i th bit of x):

$$(x_{63} \times 2^{63}) + (x_{62} \times 2^{62}) + (x_{61} \times 2^{61}) + \dots + (x_1 \times 2^1) + (x_0 \times 2^0)$$

For reasons we will shortly see, these positive numbers are called unsigned numbers.

Hardware/ Software Interface

Base 2 is not natural to human beings; we have 10 fingers and so find base 10 natural. Why didn't computers use decimal? In fact, the first commercial computer *did* offer decimal arithmetic. The problem was that the computer still used on and off signals, so a decimal digit was simply represented by several binary digits. Decimal proved so inefficient that subsequent computers reverted to all binary, converting to base 10 only for the relatively infrequent input/output events.

Keep in mind that the binary bit patterns above are simply *representatives* of numbers. Numbers really have an infinite number of digits, with almost all being 0 except for a few of the rightmost digits. We just don't normally show leading 0s.

Hardware can be designed to add, subtract, multiply, and divide these binary bit patterns. If the number that is the proper result of such operations cannot be represented by these rightmost hardware bits, *overflow* is said to have occurred. It's up to the programming language, the operating system, and the program to determine what to do if overflow occurs.

Computer programs calculate both positive and negative numbers, so we need a representation that distinguishes the positive from the negative. The most obvious solution is to add a separate sign, which conveniently can be represented in a single bit; the name for this representation is *sign and magnitude*.

Alas, sign and magnitude representation has several shortcomings. First, it's not obvious where to put the sign bit. To the right? To the left? Early computers tried both. Second, adders for sign and magnitude may need an extra step to set the sign because we can't know in advance what the proper sign will be. Finally, a separate sign bit means that sign and magnitude has both a positive and a negative zero, which can lead to problems for inattentive programmers. Because of these shortcomings, sign and magnitude representation was soon abandoned.

In the search for a more attractive alternative, the question arose as to what would be the result for unsigned numbers if we tried to subtract a large number from a small one. The answer is that it would try to borrow from a string of leading 0s, so the result would have a string of leading 1s.

Given that there was no obvious better alternative, the final solution was to pick the representation that made the hardware simple: leading 0s mean positive, and leading 1s mean negative. This convention for representing signed binary numbers is called *two's complement* representation:

```

00000000 00000000 00000000 00000000 00000000 00000000 00000000two = 0ten
00000000 00000000 00000000 00000000 00000000 00000000 00000001two = 1ten
00000000 00000000 00000000 00000000 00000000 00000000 00000010two = 2ten
...
01111111 11111111 11111111 11111111 11111111 11111111 11111111 11111101two = 9,223,372,036,854,775,805ten
01111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110two = 9,223,372,036,854,775,806ten
01111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111two = 9,223,372,036,854,775,807ten
10000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000two = - 9,223,372,036,854,775,808ten
10000000 00000000 00000000 00000000 00000000 00000000 00000000 00000001two = - 9,223,372,036,854,775,807ten
10000000 00000000 00000000 00000000 00000000 00000000 00000000 00000010two = - 9,223,372,036,854,775,806ten
...
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111101two = - 3ten
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110two = - 2ten
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111two = - 1ten

```

The positive half of the numbers, from 0 to 9,223,372,036,854,775,807_{ten} ($2^{63}-1$), use the same representation as before. The following bit pattern (1000 ... 0000_{two}) represents the most negative number -9,223,372,036,854,775,808_{ten} (-2^{63}). It is followed by a declining set of negative numbers: -9,223,372,036,854,775,807_{ten} (1000 ... 0001_{two}) down to -1_{ten} (1111 ... 1111_{two}).

Two's complement does have one negative number that has no corresponding positive number: $-9,223,372,036,854,775,808_{\text{ten}}$. Such imbalance was also a worry to the inattentive programmer, but sign and magnitude had problems for both the programmer *and* the hardware designer. Consequently, every computer today uses two's complement binary representations for signed numbers.

Two's complement representation has the advantage that all negative numbers have a 1 in the most significant bit. Thus, hardware needs to test only this bit to see if a number is positive or negative (with the number 0 is considered positive). This bit is often called the *sign bit*. By recognizing the role of the sign bit, we can represent positive and negative 64-bit numbers in terms of the bit value times a power of 2:

$$(x_{63} \times -2^{63}) + (x_{62} \times 2^{62}) + (x_{61} \times 2^{61}) + \dots + (x_1 \times 2^1) + (x_0 \times 2^0)$$

The sign bit is multiplied by -2^{63} , and the rest of the bits are then multiplied by positive versions of their respective base values.

EXAMPLE

Binary to Decimal Conversion

What is the decimal value of this 64-bit two's complement number?

11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111100_{two}

Substituting the number's bit values into the formula above:

$$\begin{aligned} & (1 \times -2^{63}) + (1 \times 2^{62}) + (1 \times 2^{61}) + \dots + (1 \times 2^1) + (0 \times 2^1) + (0 \times 2^0) \\ &= -2^{63} + 2^{62} + 2^{61} + \dots + 2^2 + 0 + 0 \\ &= -9,223,372,036,854,775,808_{\text{ten}} + 9,223,372,036,854,775,804_{\text{ten}} \\ &= -4_{\text{ten}} \end{aligned}$$

We'll see a shortcut to simplify conversion from negative to positive soon.

Just as an operation on unsigned numbers can overflow the capacity of hardware to represent the result, so can an operation on two's complement numbers. Overflow occurs when the leftmost retained bit of the binary bit pattern is not the same as the infinite number of digits to the left (the sign bit is incorrect): a 0 on the left of the bit pattern when the number is negative or a 1 when the number is positive.

ANSWER

Signed versus unsigned applies to loads as well as to arithmetic. The *function* of a signed load is to copy the sign repeatedly to fill the rest of the register—called *sign extension*—but its *purpose* is to place a correct representation of the number within that register. Unsigned loads simply fill with 0s to the left of the data, since the number represented by the bit pattern is unsigned.

When loading a 64-bit doubleword into a 64-bit register, the point is moot; signed and unsigned loads are identical. ARMv8 does offer two flavors of byte loads: *load byte* (LDURB) treats the byte as an unsigned number and thus zero-extends to fill the leftmost bits of the register, while *load byte signed* (LDURSB) works with signed integers. Since C programs almost always use bytes to represent characters rather than consider bytes as very short signed integers, LDURB is used practically exclusively for byte loads.

Hardware/ Software Interface

Unlike the signed numbers discussed above, memory addresses naturally start at 0 and continue to the largest address. Put another way, negative addresses make no sense. Thus, programs want to deal sometimes with numbers that can be positive or negative and sometimes with numbers that can be only positive. Some programming languages reflect this distinction. C, for example, names the former *integers* (declared as `long long int` in the program) and the latter *unsigned integers* (`unsigned long long int`). Some C style guides even recommend declaring the former as `signed long long int` to keep the distinction clear.

Hardware/ Software Interface

Let's examine two useful shortcuts when working with two's complement numbers. The first shortcut is a quick way to negate a two's complement binary number. Simply invert every 0 to 1 and every 1 to 0, then add one to the result. This shortcut is based on the observation that the sum of a number and its inverted representation must be $111 \dots 111_{\text{two}}$, which represents -1 . Since $x + \bar{x} = -1$, therefore $x + \bar{x} + 1 = 0$ or $\bar{x} + 1 = -x$. (We use the notation $\bar{x}$ to mean invert every bit in x from 0 to 1 and vice versa.)

Negation Shortcut

Negate 2_{ten} , and then check the result by negating -2_{ten} .

$2_{\text{ten}} = 00000000 \ 00000000 \ 00000000 \ 00000000 \ 00000000 \ 00000000 \ 00000000 \ 00000010_{\text{two}}$

EXAMPLE

ANSWER

